Node.js — Day 23

NPM & Packages ■

Long-term learning notes
These notes capture the concepts, distinctions, practical commands, examples, questions, and discoveries from Day 23 for later revision and backend development.

1. What is npm?

npm is the package manager commonly used with Node.js. It allows a project to install, manage, and keep track of external packages that the project needs.

Think of a Node.js project as a toolbox:

- Node.js built-in modules → tools already available with Node.js
- npm → the system used to get additional tools
- Packages → additional tools installed for the project

```
npm install express
```

This command tells npm to install the Express package for the current project.

2. Node.js and npm are not the same thing

Node.js	npm
A JavaScript runtime that executes JavaScript outside the browser.	A package manager used to install and manage packages for Node.js projects.

3. Creating an npm project

You created the Day 23 project using the interactive command:

```
npm init
```

npm walked through common project fields such as package name, version, description, entry point, and test command. Pressing Enter accepted the displayed defaults.

Creating the project with defaults

```
npm init -y
```

The -y option accepts the default answers automatically and creates package.json without the interactive questions.

4. What is package.json?

Think of package.json as the project's information sheet and dependency list. It contains project metadata and declares packages the project depends on.

```
{
"name": "day23",
"version": "1.0.0",
"description": "Learning NPM and Node.js packages",
"main": "index.js"
}
```

When you later install a package, npm can add it to the dependencies section of this file.

5. Dependencies

A dependency is a package that your project relies on.

```
"dependencies": {
"express": "^5.2.1"
}
```

This says: the project depends on the Express package.

6. Understanding a dependency entry

```
"dependencies": {
"express": "^5.2.1"
}
```

Part	Meaning
dependencies	The section containing packages required by the project.
express	The actual package name.
5.2.1	The version recorded for the package.
^	A version-range indicator allowing compatible updates within the same major version under npm's versioning rules.

If there are multiple entries:

```
"dependencies": {
"express": "^5.2.1",
"cors": "^2.8.5"
}
```

The project depends on 2 packages: express and cors.

7. node_modules

node_modules is the directory where npm installs packages for the project, including the dependencies required by those packages.

```
day23/
├── node_modules/
├── package.json
└── package-lock.json
```

After you ran npm install express, Express and its required packages appeared under node_modules.

8. package-lock.json

package-lock.json records the resolved dependency information and versions from an installation, including the dependency tree.

Its purpose is to help npm reproduce a consistent dependency installation across machines.

> Important: You normally do not manually edit package-lock.json. npm manages it when packages are installed, updated, or removed.

9. The three important project pieces

File / section	Easy mental model
package.json	What the project needs + project information.
package-lock.json	The resolved dependency tree and version details from installation.
node_modules	The packages currently installed locally.

10. What happens when you run npm install express?

You performed this command yourself:

```
npm install express
```

The result was visible in your project:

- Express was installed into node_modules.
- Express was added under dependencies in package.json.
- package-lock.json was created/updated with dependency details.

11. Using an installed package

After installing an external package, your JavaScript code can load it. You used CommonJS, which you already learned earlier:

```
const express = require("express");

console.log("Express loaded successfully!");
```

You ran the program and received:

```
Express loaded successfully!
```

> What did this prove?
> npm successfully installed Express, and Node.js was able to load the installed package from the project.

12. The complete npm workflow

```
npm install package
↓
package installed in node_modules
↓
package recorded in package.json
↓
dependency details recorded in package-lock.json
↓
require("package")
↓
use the package in your code
```

13. Reinstalling dependencies

If node_modules is deleted but package.json remains, you can restore the project's dependencies with:

```
npm install
```

npm reads the dependency information and installs the required packages again.

14. What if package-lock.json is also deleted?

You asked an important practical question during the lesson.

If you delete both:

```
node_modules/
package-lock.json
```

but keep package.json, you can still run:

```
npm install
```

npm can read the dependencies from package.json, install them into node_modules, and create a new package-lock.json.

The mental model

```
package.json
↓
tells npm what packages are needed
↓
npm install
↓
node_modules/ ← recreated
package-lock.json ← recreated
```

What if package.json is deleted too?

That is different. If package.json is also gone, the project's declared dependency list is gone. npm no longer has that project dependency information available from the package file.

> Key takeaway: package.json is the important project file that declares what external packages the project depends on.

15. npm init vs npm install

Command	Purpose
npm init	Create a package.json interactively.
npm init -y	Create package.json using default answers.

npm install express	Install the Express package and save it as a dependency.
npm install	Install the project's declared dependencies.

■ Long-Term Cheat Sheet

Term	Remember
npm	Node.js package manager.
npm init	Create package.json interactively.
npm init -y	Create package.json using defaults.
npm install package	Install a package for the project.
npm install	Install dependencies declared by the project.
package.json	Project information + dependency declarations.
node_modules	Installed packages and their dependencies.
package-lock.json	Resolved dependency tree and version details.
dependency	A package the project relies on.
require('express')	Load an installed package using CommonJS.

■ Day 23 Completion

DAY 23 — COMPLETE ■

Concept checks completed successfully.

Practical work completed: created an npm project, created package.json, installed Express, inspected dependencies, and successfully loaded Express with require().

Most important mental model

```
package.json → what the project needs
package-lock.json → resolved dependency details
node_modules → what is installed locally
```

Next planned topic: Middleware concepts ■■

The goal remains understanding the Node.js foundation properly before moving deeper into Express.